UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

Matemática Computacional (1AMA0726) · Ciclo 2026-20

Componentes conexas de un grafo dirigido

Proyecto (TB) · Informe final — Semana 11

Algoritmo de Kosaraju con interfaz interactiva paso a paso

Sección:	[SWxx — completar]
Grupo:	[N.° de grupo — completar]
Integrantes:	[Apellidos, Nombres — código UPC]
	[Apellidos, Nombres — código UPC]
	[Apellidos, Nombres — código UPC]
	[Apellidos, Nombres — código UPC]
	[Apellidos, Nombres — código UPC]
Docente:	[Nombre del docente]
Repositorio:	`github.com/adletgamer/Trabajo-MComputacional`

Lima, noviembre de 2026

Índice

1 Introducción **2**

2 Objetivo **2**

3 Metodología **2**

3.1 Grafos dirigidos y matriz de adyacencia 2

3.2 Componentes conexas de un grafo dirigido 3

3.3 Algoritmo de Kosaraju 3

3.4 Verificación independiente 3

3.5 Herramientas y librerías 4

4 Desarrollo del código **4**

4.1 Estructura del programa 4

4.2 Módulo `grafo.py`: el grafo dirigido 4

4.3 Módulo `algoritmo.py`: Kosaraju paso a paso 5

4.4 Módulo `interfaz.py`: interfaz gráfica 7

4.5 Validaciones y ejecutable 10

5 Diagrama de flujo **10**

6 Resultados **13**

6.1 Ejemplo manual con 8 nodos 13

6.2 Ejemplo aleatorio con 10 nodos 16

6.3 Pruebas del programa 16

6.4 Comportamiento según el tamaño del grafo 17

7 Conclusiones **18**

Referencias **19**

1. Introducción

Muchos sistemas de la vida real se representan mejor con relaciones que tienen sentido: una página web enlaza a otra, un módulo de software depende de otro, una persona sigue a otra en una red social, una calle se recorre solo en un sentido. En todos ellos aparece la misma pregunta: ¿qué elementos están tan interconectados que desde cualquiera se puede llegar a cualquier otro y volver? Responderla permite detectar dependencias circulares, agrupar elementos que se influyen mutuamente y simplificar redes grandes en estructuras más fáciles de analizar.

Este proyecto desarrolla una aplicación de escritorio que responde esa pregunta sobre grafos dirigidos pequeños, de entre 4 y 12 nodos. El programa pide la cantidad de nodos, permite construir el grafo de forma manual (eligiendo cada arista $u \to v$) o de forma aleatoria, muestra su matriz de adyacencia y ejecuta el algoritmo de Kosaraju mostrando cada decisión. El usuario avanza o retrocede por los pasos a su ritmo y observa cómo se recorren los vértices, cómo se forman las componentes y cuál es la solución final.

La idea de implementación separa el cálculo de la interfaz: el módulo del algoritmo ejecuta Kosaraju una sola vez y guarda una lista de pasos inmutables (el estado completo del algoritmo después de cada acción); la interfaz solo recorre esa lista y dibuja el paso elegido. Gracias a ello, avanzar, retroceder, reproducir automáticamente o saltar al resultado son operaciones triviales y siempre coherentes con el algoritmo real.

El informe presenta el objetivo (sección 2), la metodología y el algoritmo (sección 3), el desarrollo del código (sección 4), los diagramas de flujo (sección 5), los resultados obtenidos (sección 6) y las conclusiones (sección 7).

2. Objetivo

Desarrollar una aplicación interactiva en Python que, a partir de un grafo dirigido de n nodos ($4 \leq n \leq 12$) generado de forma manual o aleatoria, muestre su matriz de adyacencia y ejecute paso a paso el algoritmo de Kosaraju para determinar el número de componentes conexas, los nodos de cada una y su representación gráfica.

3. Metodología

3.1. Grafos dirigidos y matriz de adyacencia

Un grafo dirigido $G = (V, E)$ consta de un conjunto de vértices $V = \{1, \ldots, n\}$ y de un conjunto de aristas E formado por pares ordenados (u, v), que se dibujan como flechas $u \to v$ (Rosen, 2019). En este proyecto el grafo es simple: no admite lazos ni aristas repetidas, pero sí las dos aristas opuestas $u \to v$ y $v \to u$.

Su matriz de adyacencia A es una matriz de $n \times n$ con $A_{ij} = 1$ si existe la arista $i \to j$ y $A_{ij} = 0$ en caso contrario. A diferencia del caso no dirigido, A no es simétrica en general: la fila i lista los sucesores de i y la columna j lista sus predecesores.

3.2. Componentes conexas de un grafo dirigido

En un grafo dirigido la conexidad depende del sentido de las aristas. Dos vértices u y v pertenecen a la misma componente conexa (llamada *fuertemente conexa*) cuando existe un camino de u a v y también un camino de v a u (Cormen et al., 2022). Esta relación es de equivalencia, de modo que las componentes forman una partición de V: cada vértice pertenece exactamente a una componente, y una componente puede tener un solo vértice.

Si se ignora el sentido de las aristas se obtienen las componentes *débilmente* conexas, que son una agrupación más gruesa: toda componente fuerte está contenida en una débil. El programa resuelve el problema principal con componentes fuertes y, como información complementaria, también informa cuántas componentes débiles tiene el grafo.

3.3. Algoritmo de Kosaraju

El algoritmo, atribuido a S. R. Kosaraju y publicado por Sharir (1981), emplea dos recorridos en profundidad (DFS) y se apoya en el *grafo transpuesto* G^T, que tiene los mismos vértices que G con todas las aristas invertidas (Cormen et al., 2022, sec. 22.5). Sus etapas son:

1. **Fase 1 (DFS sobre G).** Se recorre cada vértice aún no visitado con DFS siguiendo las aristas hacia adelante. Cuando un vértice agota todos sus sucesores "termina" y recibe el siguiente número de orden de finalización $f = 1, 2, \ldots, n$.
2. **Construcción de G^T.** Se invierte cada arista $u \to v$ a $v \to u$ y se reinicia el conjunto de visitados.
3. **Fase 2 (DFS sobre G^T).** Se recorren los vértices en orden decreciente de finalización. Cada vez que se encuentra un vértice no visitado, el DFS sobre G^T desde él alcanza exactamente los vértices de una componente, que se registra como C_k.
4. **Resultado.** Al agotarse los vértices se obtienen $C_1, \ldots, C_k$; el número de componentes es k.

Por qué funciona. Si cada componente se contrae a un solo punto se obtiene un grafo sin ciclos. El vértice que termina último en la fase 1 pertenece a una componente "origen" (a la que no llega ninguna arista desde otra componente). En G^T esa componente pasa a ser "sumidero": desde ella el DFS no puede escapar a otras componentes, de modo que el árbol DFS recoge exactamente esa componente y nada más. Al repetir el razonamiento con el siguiente vértice sin asignar, se van extrayendo las demás (Cormen et al., 2022, teorema 22.16).

Complejidad. Cada fase recorre cada vértice y cada arista una vez; con listas de sucesores el costo es $O(n + m)$, donde $m = |E|$. Para $n \leq 12$ el tiempo es imperceptible, por lo que el costo de la interfaz (dibujar cada paso) domina sobre el cálculo.

3.4. Verificación independiente

Para no confiar ciegamente en el algoritmo, el programa comprueba su resultado con un método distinto: calcula el cierre transitivo del grafo (algoritmo de Warshall, $O(n^3)$) y verifica que (a) las compo-

nentes formen una partición de V, (b) todo par de vértices de una misma componente sea mutuamente alcanzable y (c) ningún par de vértices de componentes distintas lo sea.

3.5. Herramientas y librerías

El programa se escribió en Python 3 usando únicamente la biblioteca estándar; no requiere instalar paquetes. La Tabla 1 resume su uso. El ejecutable de Windows se genera con PyInstaller (script `construir_exe.bat`); esta herramienta solo se usa para empaquetar y no forma parte del código del proyecto.

Tabla 1
Librerías utilizadas en el programa

Librería	Uso en el proyecto
`tkinter`, `tkinter.ttk`	Ventana, lienzo (`Canvas`) para dibujar el grafo, botones, cuadros de texto y diálogos de error.
`random`	Sorteo de las aristas del grafo aleatorio.
`math`	Posición circular de los vértices y geometría de las flechas.
`dataclasses`	Clases inmutables `Paso` y `Certificado`.
`unittest`	Pruebas automáticas (14 pruebas).

Nota. Todas pertenecen a la biblioteca estándar de Python (Python Software Foundation, 2024).

4. Desarrollo del código

4.1. Estructura del programa

El código está dividido en cuatro módulos con responsabilidades separadas (Tabla 2). La capa de datos y el algoritmo no dependen de la interfaz, lo que permite probarlos de forma automática.

Tabla 2
Módulos del programa

Archivo	Líneas	Responsabilidad
`main.py`	8	Punto de entrada: crea y ejecuta la ventana.
`grafo.py`	111	Grafo dirigido: aristas, matriz de adyacencia, validaciones y generación aleatoria.
`algoritmo.py`	213	Kosaraju paso a paso, componentes débiles, aristas entre componentes y verificación.
`interfaz.py`	590	Pantallas, dibujo del grafo con flechas, matriz, panel de solución y navegación.
`tests/test_proyecto.py`	180	14 pruebas automáticas.

Nota. Conteo de líneas de la versión final del repositorio.

4.2. Módulo `grafo.py`: el grafo dirigido

La clase `Grafo` guarda, para cada vértice, el conjunto de sus sucesores: la arista $u \to v$ existe si v está en `adyacencia[u]`. Agregar una arista solo modifica el conjunto de u (en un grafo no dirigido habría

que modificar también el de v), y la matriz de adyacencia se construye a partir de esos conjuntos. El constructor rechaza cualquier n fuera de $[4, 12]$ y `agregar_arista` rechaza vértices fuera de rango y lazos, lo que sostiene las validaciones de la interfaz.

```
34      def agregar_arista(self, u: int, v: int) -> bool:
35          """Agrega u → v. Devuelve False si ya existía."""
36          self._validar_extremos(u, v)
37          if v in self.adyacencia[u]:
38              return False
39          self.adyacencia[u].add(v)
40          return True
```

Fragmento 1. Método `agregar_arista`: solo se actualiza el conjunto de sucesores de `u`.

La generación aleatoria sortea entre n y $1{,}6\,n$ pares ordenados (u, v) sin repetir y sin lazos. Con aristas dirigidas tomadas al azar, un grafo muy denso casi siempre es una sola componente y uno muy ralo es un conjunto de vértices aislados; por eso el rango elegido es intermedio y, además, se descartan (hasta 50 intentos) los sorteos poco ilustrativos: una sola componente, o menos de 3 vértices dentro de componentes con ciclos. Cada llamada usa un generador nuevo sembrado por el sistema, así que cada demostración produce un grafo distinto.

```
76      @classmethod
77      def generar_aleatorio(cls, n: int, rng: random.Random | None = None) -> "Grafo":
78          """Grafo dirigido aleatorio (entre n y ~1.6n aristas sorteadas al azar).
79
80          Cada llamada produce un grafo distinto (si no se pasa ``rng`` se usa un
81          generador nuevo sembrado por el sistema). Para que la demostración sea
82          ilustrativa se descartan, hasta 50 veces, los sorteos cuyo resultado es
83          poco ilustrativo: una sola componente, o menos de 3 vértices dentro de
84          componentes con ciclos.
85          """
86          rng = rng or random.Random()
87          pares = [(u, v) for u in range(1, n + 1) for v in range(1, n + 1) if u != v]
88          minimo = max(4, n)
89          maximo = min(len(pares), max(minimo, round(1.6 * n)))
90          grafo = cls(n)
91          for _ in range(50):
92              grafo = cls(n)
93              for u, v in rng.sample(pares, rng.randint(minimo, maximo)):
94                  grafo.agregar_arista(u, v)
95              if grafo._es_interesante():
96                  break
97          return grafo
```

Fragmento 2. Generación aleatoria del grafo dirigido.

4.3. Módulo `algoritmo.py`: Kosaraju paso a paso

La función `componentes_fuertes_pasos` implementa los dos recorridos con una sola función recursiva `dfs`. En la fase 1 explora los sucesores de cada vértice; en la fase 2 explora sus predecesores, que equivale a recorrer G^T sin construirlo explícitamente. Cada acción llama a `guardar`, que agrega a la lista un objeto `Paso` con una fotografía completa del estado (fase, vértices visitados, pila de recursión, orden

de finalización, arista examinada, árbol DFS, componentes completas). Como los pasos son inmutables, la interfaz puede mostrarlos en cualquier orden sin riesgo de alterar el algoritmo.

```
89      def dfs(u: int) -> None:
90          visitados.add(u)
91          pila.append(u)
92          if fase == FASE_DFS_TRANSPUESTO:
93              componente.append(u)
94              guardar(f"Se visita {u} sobre G^T: entra en la componente actual.", u)
95          else:
96              guardar(f"Se visita {u}: entra en la pila de recursión del DFS.", u)
97          vecinos = grafo.sucesores(u) if fase == FASE_DFS_G else grafo.predecesores(u)
98          for v in vecinos:
99              if v not in visitados:
100                 arbol.append((u, v))
101                 guardar(f"Se examina {u}→{v}: {v} no estaba visitado, se desciende a {v}.",
102                         u, (u, v), "arbol")
103                 dfs(v)
104             else:
105                 guardar(f"Se examina {u}→{v}: {v} ya estaba visitado, se ignora.",
106                         u, (u, v), "visitado")
107         pila.pop()
108         if fase == FASE_DFS_G:
109             terminados.append(u)
110             guardar(f"{u} no tiene más sucesores por explorar: termina con orden de "
111                     f"finalización #{len(terminados)}.", u)
112         else:
113             guardar(f"{u} no tiene más predecesores por explorar: se cierra su rama.", u)
```

Fragmento 3. Recorrido DFS recursivo compartido por las dos fases.

Las dos fases se encadenan en el cuerpo de la función: la fase 1 llama a `dfs` desde cada vértice sin visitar y deja el orden de finalización en la lista `terminados`; la fase 2 recorre esa lista en sentido inverso y cada llamada `dfs(inicio)` completa una componente (líneas 131–140).

```
115     # ---- Fase 1: DFS sobre G, registrando el orden de finalización.
116     guardar("Estado inicial. Fase 1: DFS sobre G para obtener el orden de finalización de los vértices.")
117     for inicio in range(1, grafo.n + 1):
118         if inicio in visitados:
119             continue
120         guardar(f"Fase 1: se inicia un nuevo árbol DFS desde {inicio} (aún sin visitar).", inicio)
121         dfs(inicio)
122     orden_fase2 = list(reversed(terminados))
123     guardar("Fin de la fase 1. Orden de finalización: " + ", ".join(map(str, terminados)) + ".")
124
125     # ---- Fase 2: DFS sobre G^T en orden inverso de finalización.
126     fase = FASE_DFS_TRANSPUESTO
127     visitados.clear()
128     arbol.clear()
129     guardar("Fase 2: se invierten todas las aristas (G^T) y se procesan los vértices en orden inverso "
130             "al de finalización: " + ", ".join(map(str, orden_fase2)) + ".")
131     for inicio in orden_fase2:
132         if inicio in visitados:
133             continue
134         componente.clear()
135         guardar(f"Fase 2: {inicio} es el siguiente vértice sin asignar; el DFS sobre G^T desde {inicio} "
136                 f"recogerá una componente completa.", inicio)
137         dfs(inicio)
138         completas.append(sorted(componente))
```

```
        guardar(f"Componente C{len(completas)} completa: {formatear_componente(sorted(componente))}.")
        componente.clear()
```

Fragmento 4. Fases 1 y 2 del algoritmo de Kosaraju.

La función `verificar_componentes` contrasta el resultado con el cierre transitivo calculado por el algoritmo de Warshall, según lo descrito en la sección 3.4.

```
def verificar_componentes(grafo: Grafo, componentes: list[list[int]]) -> Certificado:
    """Comprueba el resultado con un método independiente (cierre transitivo).

    * los vértices quedan particionados,
    * dentro de una componente todo par es mutuamente alcanzable,
    * entre componentes distintas nunca hay alcanzabilidad mutua.
    """
    vertices = [v for c in componentes for v in c]
    cubiertos = len(vertices) == grafo.n and set(vertices) == set(range(1, grafo.n + 1))
    if not cubiertos:
        return Certificado(False, False, False)
    r = _alcanzables(grafo)
    grupo = {v: i for i, c in enumerate(componentes) for v in c}
    fuertes = True
    sin_ciclos = True
    for u in range(1, grafo.n + 1):
        for v in range(u + 1, grafo.n + 1):
            mutuo = r[u - 1][v - 1] and r[v - 1][u - 1]
            if grupo[u] == grupo[v] and not mutuo:
                fuertes = False
            if grupo[u] != grupo[v] and mutuo:
                sin_ciclos = False
    return Certificado(True, fuertes, sin_ciclos)
```

Fragmento 5. Verificación independiente de las componentes.

4.4. Módulo `interfaz.py`: interfaz gráfica

La ventana tiene dos pantallas. En la *pantalla de inicio* (Figura 1) el usuario escribe n, que se valida (entero entre 4 y 12; cualquier otro valor muestra un mensaje de error) y elige entre generar el grafo de forma manual o aleatoria. En el *área de trabajo* (Figuras 2 y siguientes) se distinguen cuatro zonas:

- **Lienzo del grafo** (izquierda): vértices en círculo y aristas con flecha. Cuando existen $u \to v$ y $v \to u$, cada una se dibuja curva hacia su lado para que no se superpongan. Un código de colores muestra el estado de cada vértice (sin visitar, en la pila del DFS, actual, terminado, componente completa) y las aristas se diferencian entre árbol DFS (continua), ya visitada (discontinua) y arista que se examina (coral).
- **Panel lateral**: controles de construcción (modo manual: elegir u, elegir v, *Agregar*, *Quitar*, *Limpiar*, *Analizar*; modo aleatorio: *Generar otro grafo*), la matriz de adyacencia y la solución.
- **Barra de navegación** (abajo): *Anterior*, *Siguiente*, *Reproducir/Pausar*, *Ver resultado* y volver al primer paso.
- **Estado del algoritmo** (abajo): explicación del paso actual, pila del DFS, vértices visitados, orden de finalización (o vértices por procesar en la fase 2) y componentes formadas.

En la fase 2 el lienzo dibuja G^{T} (aristas invertidas) para que se vea exactamente el grafo sobre el que se recorre, y en el paso final vuelve a mostrar G con un color por componente. La navegación es solo cambiar un índice sobre la lista de pasos y volver a dibujar, con lo que "Anterior" es tan simple como "Siguiente".

Figura 1

Pantalla de inicio del programa



Nota. Se ingresa la cantidad de nodos ($4 \leq n \leq 12$) y se elige el modo de generación. Captura de pantalla del programa desarrollado.

Figura 2

Construcción manual de un grafo dirigido



Nota. El usuario elige u y v y pulsa *Agregar*; el grafo, la lista de aristas y la matriz de adyacencia se actualizan al instante. Captura de pantalla del programa desarrollado.

El trazado de flechas es la parte geométrica más delicada de la interfaz: debe acortar cada flecha para que termine en el borde del círculo del vértice y, si existe la arista opuesta, curvarla.

```
570     def _flecha(self, p, q, radio, color, grosor, dash, curva: bool) -> None:
571         (x1, y1), (x2, y2) = p, q
572         dx, dy = x2 - x1, y2 - y1
573         largo = math.hypot(dx, dy) or 1
574         ux, uy = dx / largo, dy / largo
575         if curva:
576             # Si existen u→v y v→u, cada una se curva hacia su lado para no solaparse.
577             nx, ny = -uy, ux
578             cx, cy = (x1 + x2) / 2 + nx * 0.16 * largo, (y1 + y2) / 2 + ny * 0.16 * largo
579             ax, ay = cx - x1, cy - y1
580             la = math.hypot(ax, ay) or 1
581             bx, by = cx - x2, cy - y2
582             lb = math.hypot(bx, by) or 1
583             inicio = (x1 + ax / la * radio, y1 + ay / la * radio)
584             fin = (x2 + bx / lb * (radio + 2), y2 + by / lb * (radio + 2))
585             puntos = (*inicio, cx, cy, *fin)
586             self.lienzo.create_line(*puntos, smooth=True, fill=color, width=grosor, dash=dash,
587                                     arrow=tk.LAST, arrowshape=(11, 13, 5))
588         else:
589             self.lienzo.create_line(x1 + ux * radio, y1 + uy * radio, x2 - ux * (radio + 2), y2 - uy * (radio + 2),
590                                     fill=color, width=grosor, dash=dash, arrow=tk.LAST, arrowshape=(11, 13, 5))
```

Fragmento 6. Trazado de una arista dirigida (recta o curva) con punta de flecha.

4.5. Validaciones y ejecutable

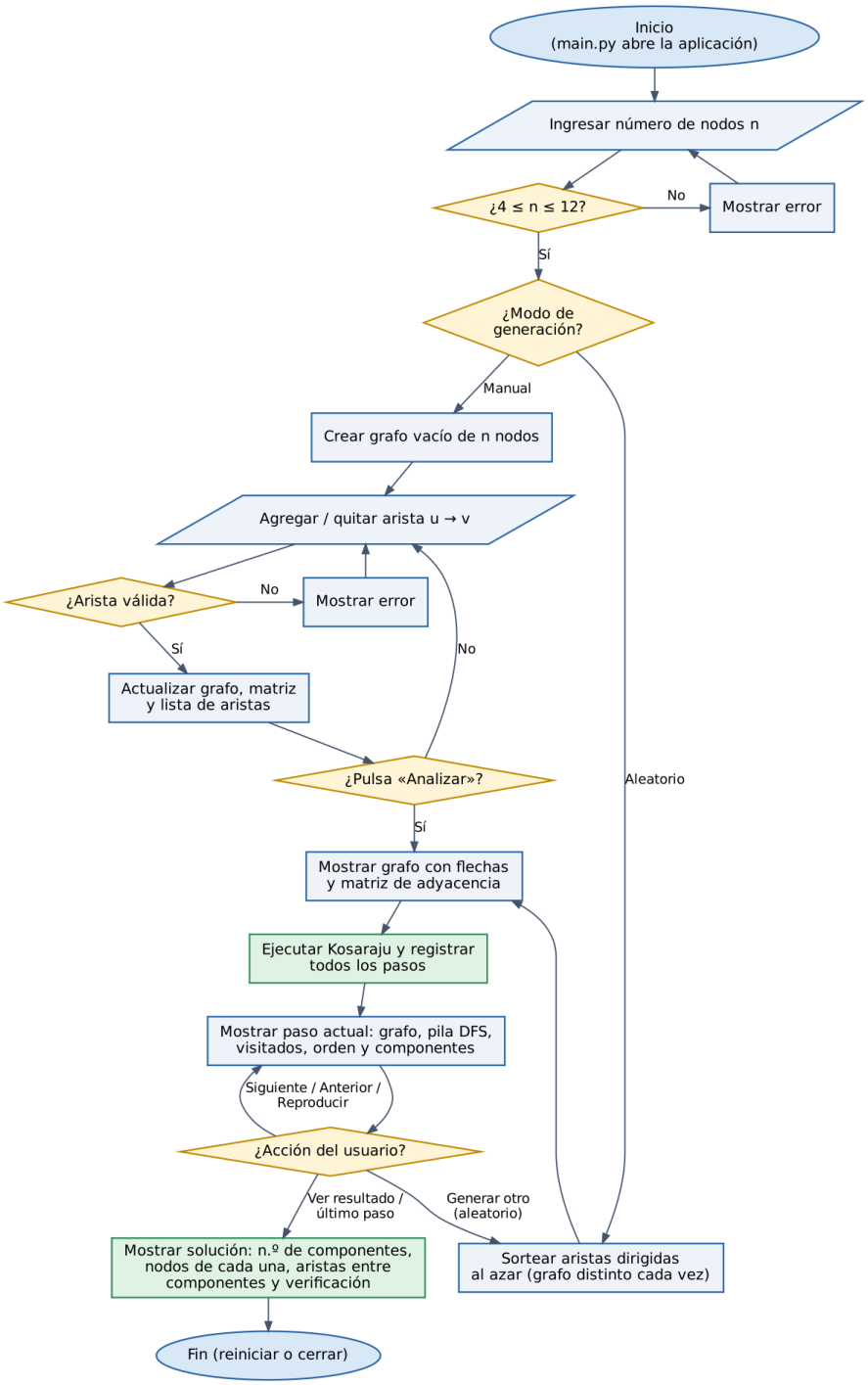
El programa controla las entradas: n vacío, no numérico o fuera de $[4, 12]$; aristas con lazos, vértices fuera de rango o repetidas (se informa que ya existe) y eliminación de aristas inexistentes. Para entregar un ejecutable sin requerir Python, el archivo `construir_exe.bat` empaqueta `main.py` con PyInstaller y genera `dist\ComponentesConexas.exe`.

5. Diagrama de flujo

La Figura 3 muestra el flujo completo del programa, desde que se abre la aplicación hasta que se presenta la solución, y la Figura 4 detalla el flujo interno del algoritmo de Kosaraju (módulo `algoritmo.py`). Ambos diagramas siguen el comportamiento real del código: los rombos de validación corresponden a las comprobaciones de `_leer_n` y `agregar_arista`, y el bucle central del programa corresponde a los botones de navegación.

Figura 3

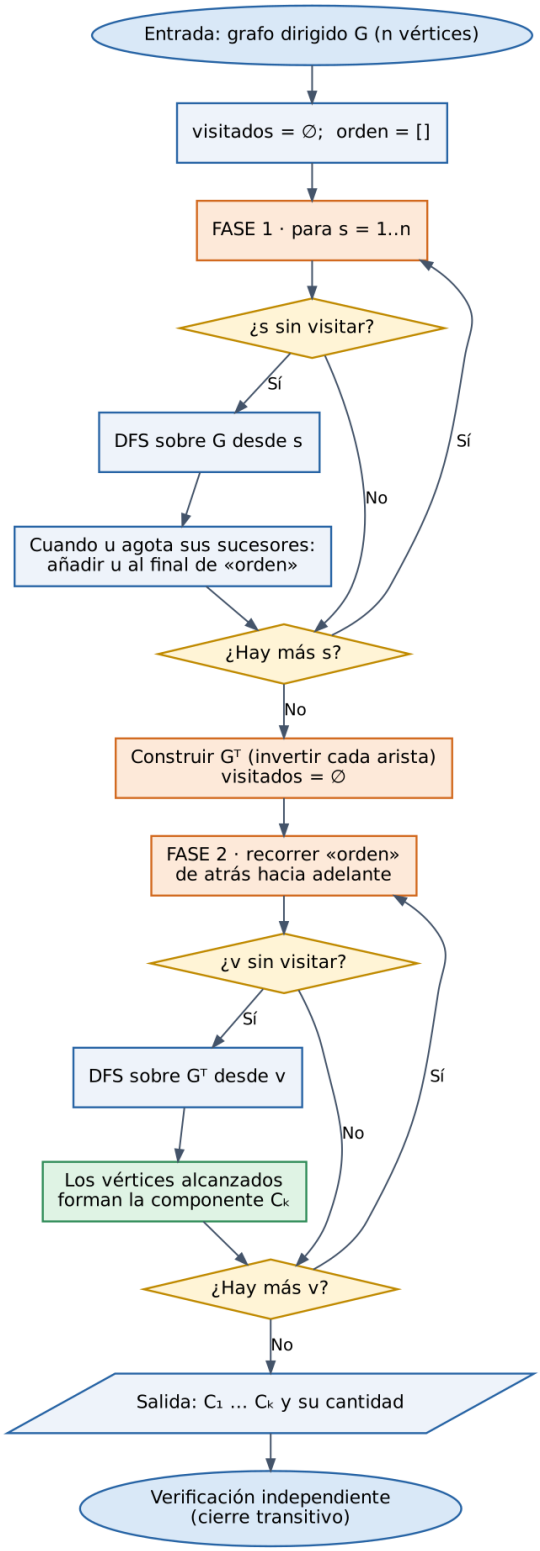
Diagrama de flujo del programa



Nota. Elaboración propia. Los óvalos indican inicio y fin, los paralelogramos entradas del usuario, los rectángulos procesos y los rombos decisiones.

Figura 4

Diagrama de flujo del algoritmo de Kosaraju



Nota. Elaboración propia, a partir de Cormen et al. (2022, sec. 22.5). Las cajas anaranjadas marcan los cambios de fase.

6. Resultados

6.1. Ejemplo manual con 8 nodos

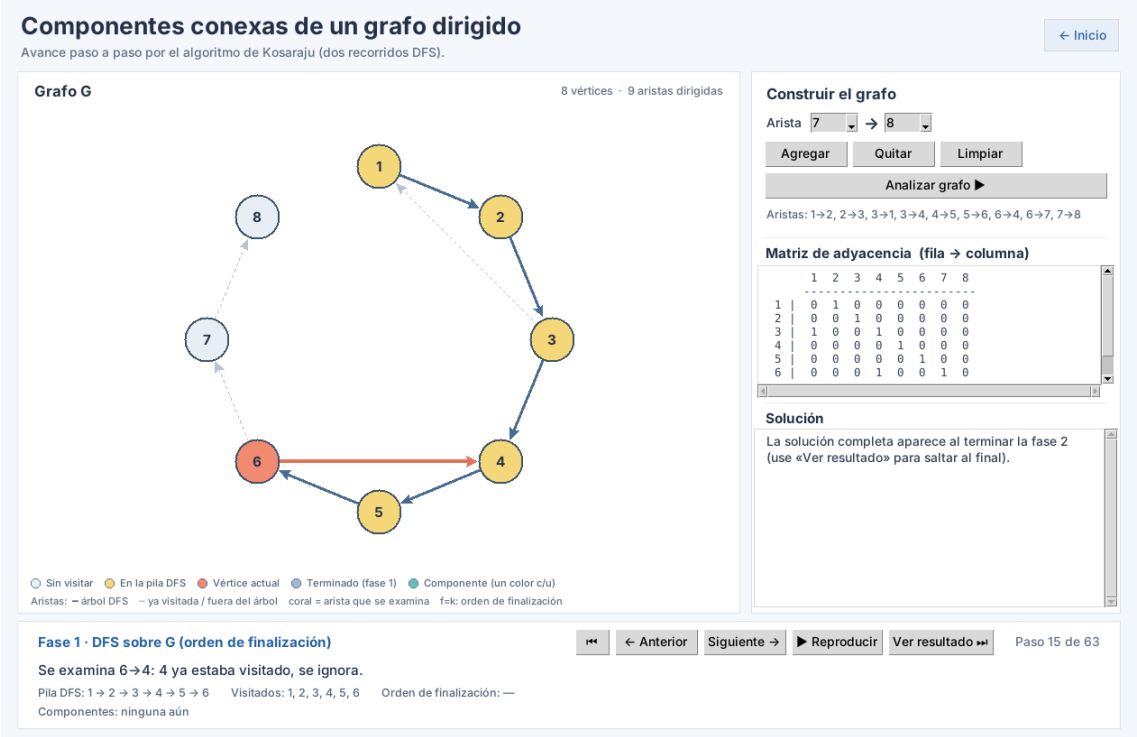
Se construyó manualmente un grafo de $n = 8$ nodos con las aristas $1 \to 2$, $2 \to 3$, $3 \to 1$, $3 \to 4$, $4 \to 5$, $5 \to 6$, $6 \to 4$, $6 \to 7$ y $7 \to 8$ (Figura 2). El programa muestra su matriz de adyacencia, que no es simétrica:

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Al analizar el grafo se registran 63 pasos: 28 de la fase 1, 34 de la fase 2 y el paso final. Las Figuras 5–8 muestran momentos representativos de la ejecución.

Figura 5

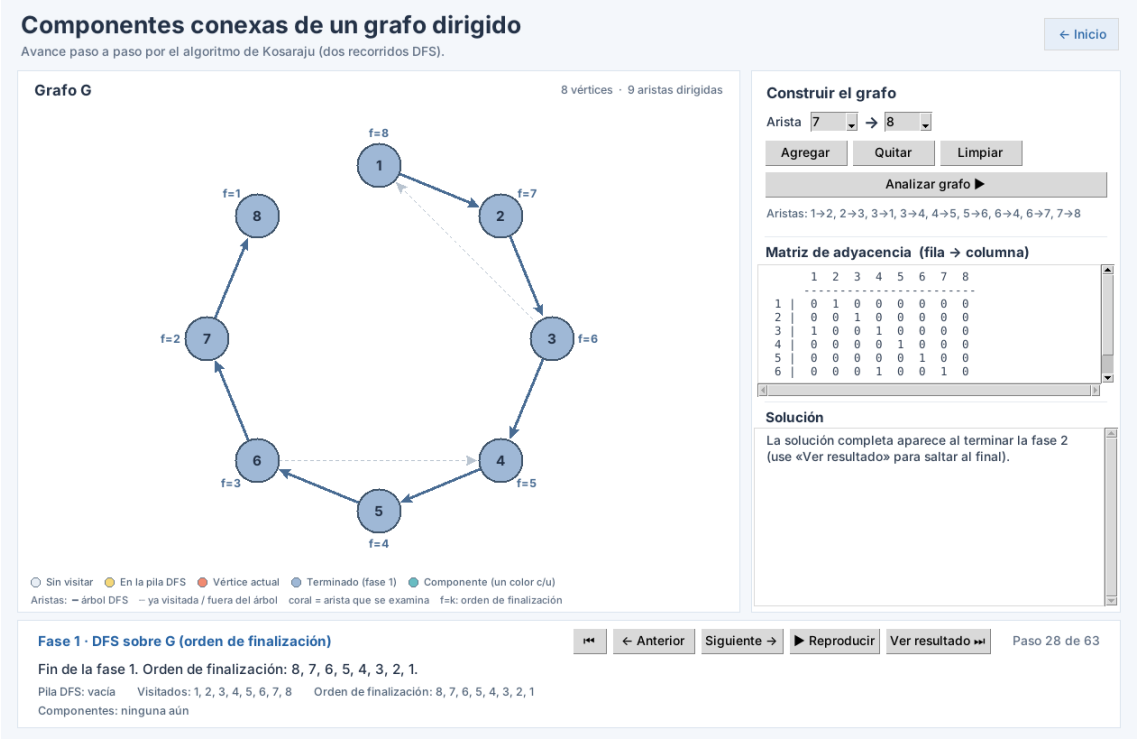
Fase 1 en curso: DFS sobre G



Nota. Paso 15 de 63. El DFS partió de 1 y descendió por 2, 3, 4, 5 y 6; se examina la arista $6 \to 4$ y se ignora porque 4 ya está en la pila. Captura de pantalla del programa.

Figura 6

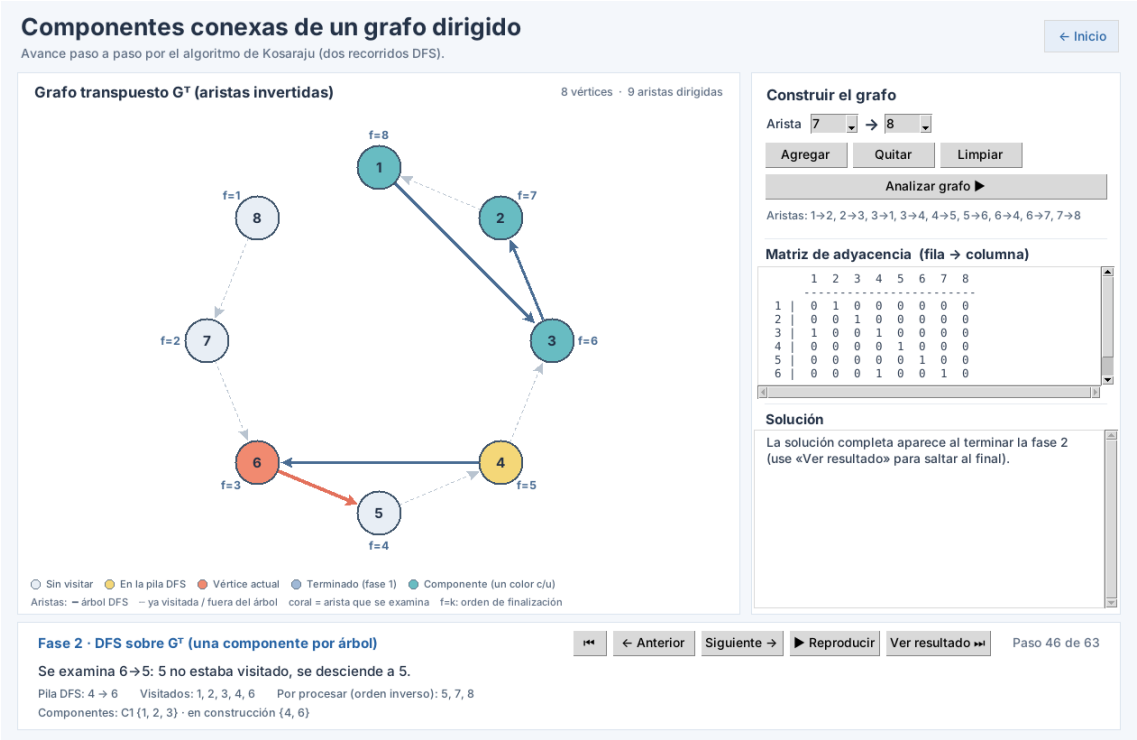
Fin de la fase 1: orden de finalización



Nota. Cada vértice muestra su orden de finalización $f = k$. El orden resultante es 8, 7, 6, 5, 4, 3, 2, 1. Captura de pantalla del programa.

Figura 7

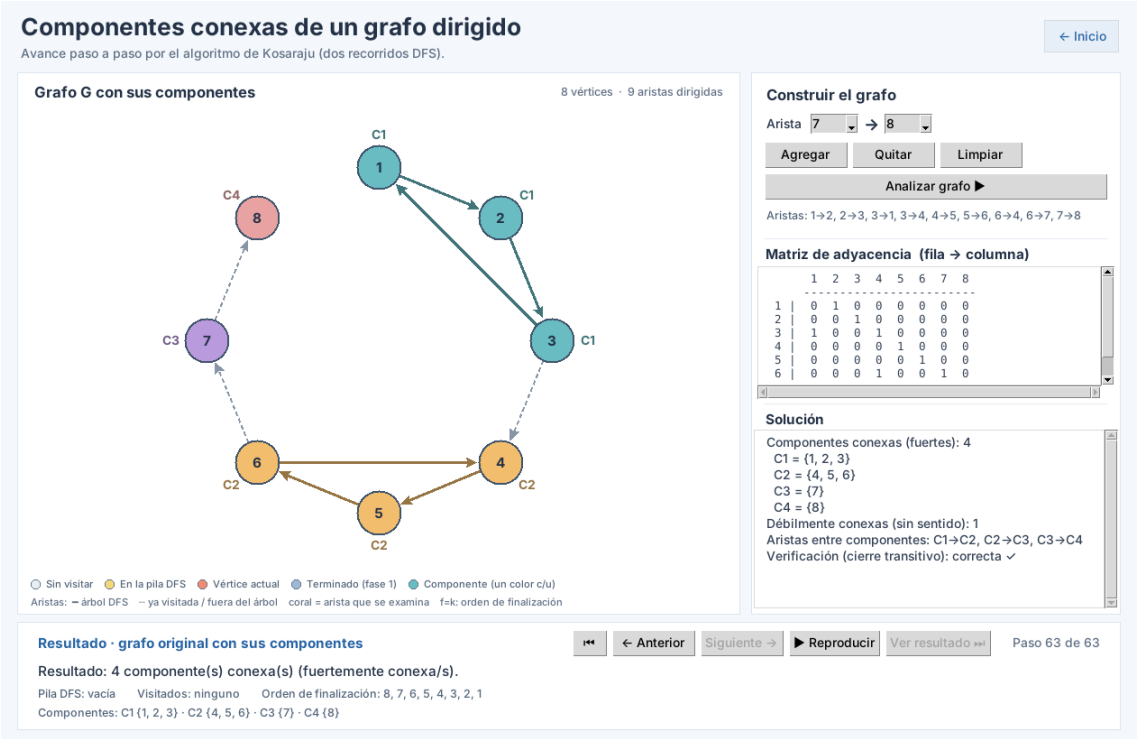
Fase 2 en curso: DFS sobre el grafo transpuesto



Nota. Las aristas se muestran invertidas. Los vértices se procesan en el orden 1, 2, 3, …; la componente $\{1, 2, 3\}$ ya está completa. Captura de pantalla del programa.

Figura 8

Resultado final del ejemplo manual



Nota. Se encontraron 4 componentes conexas, cada una con un color. Las aristas entre componentes se dibujan discontinuas. Captura de pantalla del programa.

La Tabla 3 resume la traza. En la fase 2 se procesan los vértices en orden inverso de finalización (1, 2, 3, ..., 8); cada vez que aparece uno no asignado, el DFS sobre G^T recoge su componente completa. Por ejemplo, desde 4 se llega por G^T a 6 y a 5 (porque $6 \to 4$ y $5 \to 6$), pero no a 3, ya asignado a C_1.

Tabla 3

Traza de la fase 2 del ejemplo manual

Raíz en G^T	Vértices alcanzados	Componente	Explicación
1	1, 3, 2	$C_1 = \{1, 2, 3\}$	Ciclo $1 \to 2 \to 3 \to 1$
4	4, 6, 5	$C_2 = \{4, 5, 6\}$	Ciclo $4 \to 5 \to 6 \to 4$
7	7	$C_3 = \{7\}$	Solo llega $6 \to 7$, 6 ya asignado
8	8	$C_4 = \{8\}$	Solo llega $7 \to 8$, 7 ya asignado

Nota. Los vértices se procesan en el orden inverso de finalización: 1, 2, 3, 4, 5, 6, 7, 8.

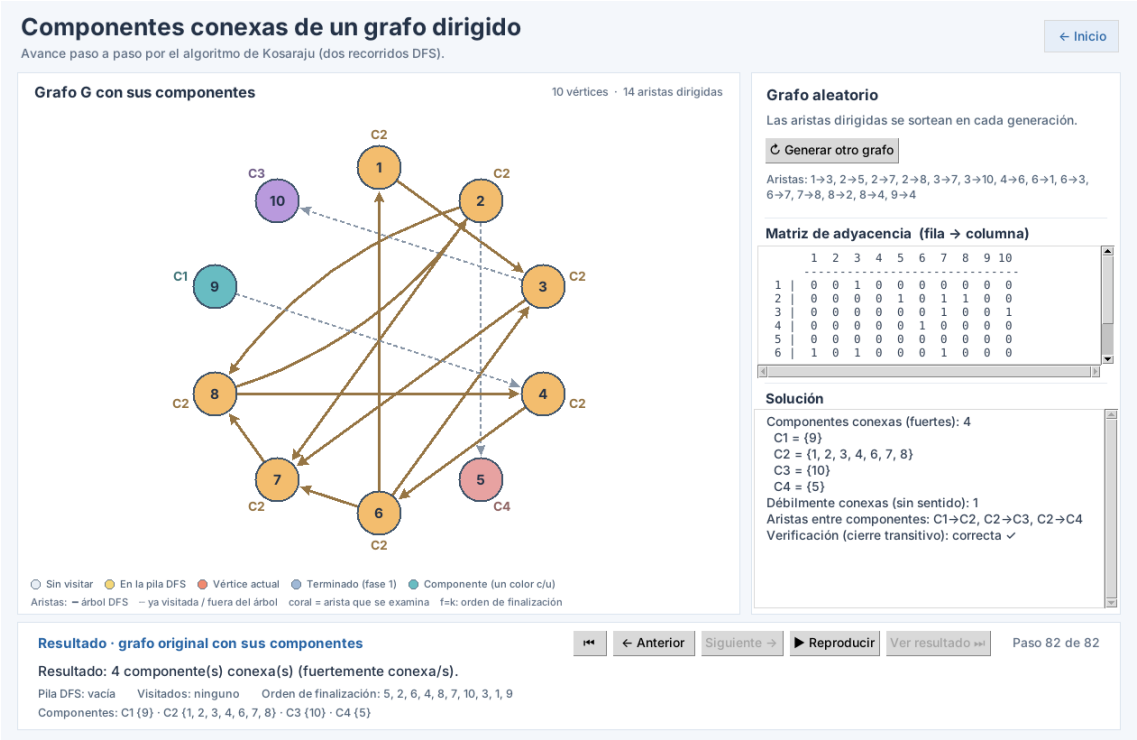
Interpretación. La solución indica **4 componentes conexas**: $\{1, 2, 3\}$, $\{4, 5, 6\}$, $\{7\}$ y $\{8\}$, con aristas entre ellas $C_1 \to C_2$, $C_2 \to C_3$ y $C_3 \to C_4$. Hay dos ciclos (que forman las dos componentes grandes) y dos vértices que no pertenecen a ningún ciclo. El ejemplo muestra la diferencia entre conexidad débil y fuerte: si se ignora el sentido de las flechas, el grafo es una sola pieza (1 componente débil), pero como no hay forma de volver de 4 a 1, ni de 7 a 6, solo hay componentes fuertes más pequeñas. La verificación con el cierre transitivo confirmó la partición.

6.2. Ejemplo aleatorio con 10 nodos

En modo aleatorio el programa sortea las aristas y deja el análisis listo para recorrerlo. La Figura 9 corresponde a una generación de $n = 10$ con 14 aristas dirigidas (cada generación produce un grafo distinto). El resultado fue de **4 componentes**: $C_1 = \{9\}$, $C_2 = \{1, 2, 3, 4, 6, 7, 8\}$, $C_3 = \{10\}$ y $C_4 = \{5\}$, con aristas entre componentes $C_1 \to C_2$, $C_2 \to C_3$ y $C_2 \to C_4$ y una sola componente débil. En el diagrama se observa que los siete vértices de C_2 se alcanzan entre sí a través de varios ciclos, mientras que 9, 10 y 5 solo tienen aristas de entrada o de salida hacia C_2 y por eso quedan aislados como componentes unitarias.

Figura 9

Resultado de un grafo aleatorio de 10 nodos



Nota. Panel *Solución* con el número de componentes, los nodos de cada una, las aristas entre componentes y la verificación. Captura de pantalla del programa.

6.3. Pruebas del programa

Se escribieron 14 pruebas automáticas con `unittest`; todas se cumplen (Tabla 4). La prueba más exigente compara Kosaraju con un cálculo independiente por alcanzabilidad mutua en 400 grafos aleatorios con n entre 4 y 12, y coincidió en todos. Además se ejecutó la interfaz de forma automática (pantalla virtual): validación de n inválido (3, 13, "abc" y vacío producen mensaje de error), flujo completo manual y aleatorio, reproducción automática hasta el último paso y 135 generaciones aleatorias sucesivas (15 por cada n de 4 a 12) con análisis y regeneración, sin errores.

Tabla 4

Pruebas automáticas del proyecto

Qué se comprueba	Resultado
Matriz dirigida no simétrica; $u \to v$ distinto de $v \to u$	Correcto
Eliminar una arista no elimina la opuesta	Correcto
Validación de n (3, 13, `True`, texto), lazos y vértices fuera de rango	Se rechazan
Generación aleatoria: rango, sin lazos ni duplicados, más de 200 grafos distintos	Correcto
Generación aleatoria ilustrativa (>285 de 300 con ≥ 2 componentes y alguna con ciclos)	Correcto
Misma semilla, mismo grafo (reproducibilidad)	Correcto
Ejemplo conocido, ciclo completo, camino dirigido, grafo sin aristas	Correcto
Kosaraju contra fuerza bruta en 400 grafos aleatorios y verificación	400/400
Pasos consistentes en 60 grafos (fases ordenadas, aristas existentes, componentes crecientes)	Correcto
El verificador detecta particiones erróneas	Correcto
Aristas entre componentes y componentes débiles	Correcto

Nota. 14 pruebas en total; el archivo es `tests/test_proyecto.py`.

6.4. Comportamiento según el tamaño del grafo

Para cada n de 4 a 12 se generaron 200 grafos aleatorios, se ejecutó el algoritmo y se verificó el resultado (Tabla 5, Figura 10). En los 1 800 grafos la verificación independiente fue correcta. El número de pasos crece de forma lineal con el tamaño del grafo. En efecto, cada ejecución registra $4n+2m+2k+r+4$ pasos, donde m es la cantidad de aristas, k el número de componentes y r la cantidad de árboles DFS de la fase 1 (por ejemplo, $n = 8$, $m = 9$, $k = 4$, $r = 1$ da $32+18+8+1+4 = 63$ pasos, como en el ejemplo manual); esta fórmula coincidió con el registro del programa en 500 grafos aleatorios adicionales. Esto es coherente con la complejidad $O(n + m)$ del algoritmo. El tiempo de cálculo es del orden de décimas de milisegundo, por lo que el usuario nunca percibe espera: el ritmo lo marca la navegación. Con $n = 12$, la mayor componente promedia alrededor de 4,7 vértices de 12, así que los grafos aleatorios mezclan componentes grandes con vértices aislados, lo que vuelve las demostraciones variadas.

Tabla 5

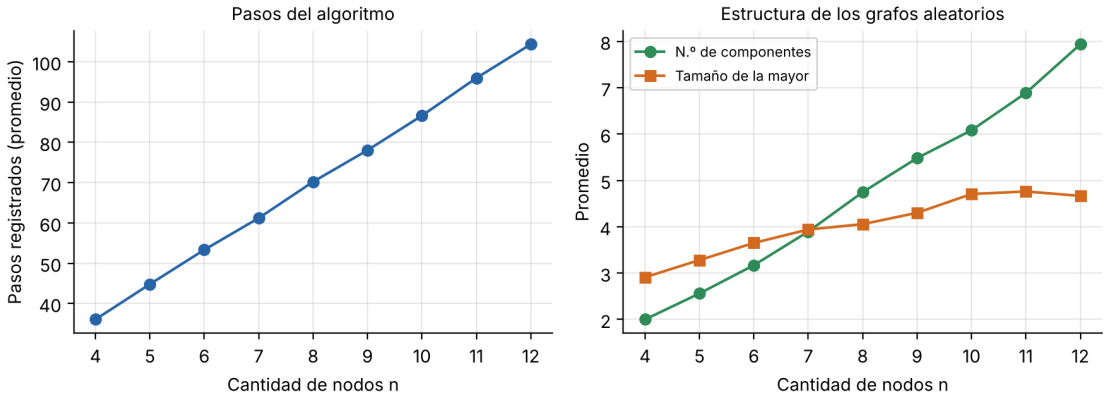
Resultados de 200 grafos aleatorios por cada cantidad de nodos

n	Aristas	Componentes	Mayor comp.	Pasos	Verificados	ms/ejec.
4	5.3	2.0	2.9	36.0	200/200	0.13
5	6.9	2.6	3.3	44.8	200/200	0.16
6	8.5	3.2	3.6	53.3	200/200	0.19
7	9.5	3.9	3.9	61.2	200/200	0.22
8	10.9	4.7	4.1	70.2	200/200	0.25
9	11.9	5.5	4.3	78.1	200/200	0.33
10	13.4	6.1	4.7	86.6	200/200	0.37
11	15.1	6.9	4.8	96.0	200/200	0.41
12	16.0	7.9	4.7	104.4	200/200	0.43

Nota. Valores promedio. *ms/ejec.* es el tiempo de generar el grafo y ejecutar el algoritmo (sin dibujar) en el entorno de pruebas, con Python 3.13.

Figura 10

Pasos del algoritmo y estructura de los grafos según n



Nota. Promedio de 200 grafos aleatorios por cada n. Elaboración propia.

7. Conclusiones

- Se cumplió el objetivo general: se desarrolló una aplicación en Python que pide n ($4 \leq n \leq 12$), construye el grafo dirigido de forma manual o aleatoria, muestra su matriz de adyacencia y ejecuta Kosaraju paso a paso, entregando el número de componentes, los nodos de cada una y su representación gráfica. Los dos ejemplos de la sección 6 (4 componentes en cada caso) lo evidencian.
- Trabajar con grafos dirigidos cambia el problema: la conexidad exige alcanzabilidad en ambos sentidos. El ejemplo manual lo demuestra: un grafo débilmente conexo de 8 vértices resultó tener 4 componentes conexas, porque no existe camino de regreso entre ellas.
- El algoritmo de Kosaraju se comprende mejor al verlo: mostrar el orden de finalización de la fase 1 y dibujar G^T en la fase 2 hace visible por qué cada árbol DFS de la segunda fase es una componente completa.

- Separar el cálculo de la interfaz (el algoritmo genera una lista de pasos inmutables) hizo posible retroceder, reproducir y saltar al resultado sin errores, y permitió probar el algoritmo de forma automática. Esta decisión de diseño fue la más rentable del proyecto.
- La solución es confiable: las 14 pruebas pasan, Kosaraju coincide con un cálculo independiente en 400 grafos aleatorios y la verificación por cierre transitivo fue correcta en los 1 800 grafos de la Tabla 5.
- Limitaciones y trabajo futuro: el trazado circular es adecuado hasta 12 vértices pero no escalaría a grafos grandes; también se podría permitir arrastrar los vértices, guardar y cargar grafos desde archivo y comparar Kosaraju con el algoritmo de Tarjan, que lo resuelve con un solo DFS.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Python Software Foundation. (2024). *tkinter — Python interface to Tcl/Tk*. Python 3 Documentation. https://docs.python.org/3/library/tkinter.html

Rosen, K. H. (2019). *Discrete mathematics and its applications* (8.ª ed.). McGraw-Hill Education.

Sharir, M. (1981). A strong-connectivity algorithm and its applications in data flow analysis. *Computers & Mathematics with Applications*, *7*(1), 67–72. https://doi.org/10.1016/0898-1221(81)90008-0

Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, *1*(2), 146–160. https://doi.org/10.1137/0201010